

# EDUCATIONAL PRACTICE MINI-PROJECT REPORT

## Title Page

Project title: House Price Prediction Using Machine Learning

Academic group: IT-2512

Team name: Defence

Project track: Machine Learning

Project topic: House price prediction

## Team Members and Contributions

| No. | Full Name         | Contribution                                                                                                                                                  |
|-----|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | Mikhail Gulyayev  | Leader / Project management, Exploratory Data Analysis (EDA), data preprocessing, Linear Regression model implementation, CLI development, report preparation |
| 2   | Beknur Abdykhamit | Member / Dataset research, model testing, documentation, model evaluation, create notebook , Exploratory Data Analysis (EDA), report preparation              |

## Project Overview

This project aims to predict house prices using machine learning. The USA Housing dataset was analyzed and used to train a Linear Regression model. The project includes data preprocessing, exploratory data analysis (EDA), model training, evaluation, and a CLI application that allows users to predict house prices based on housing characteristics.

## Problem Statement

The project solves the problem of calculating house prices. It helps predict the price of a house based on factors such as average income, house age, number of rooms, number of bedrooms, and population using a machine learning model.

## Tools and Technologies

Programming Language:

Python

Libraries:

- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-Learn
- Jupyter Notebook

Tools:

- Visual Studio Code (OSS Code)
- GitHub

Dataset:

USA Housing Dataset

<https://www.kaggle.com/datasets/vedavyasv/usa-housing/data>

## Main Features / Model Description

Exploratory Data Analysis (EDA)

- Size of dataset: 5,000 records and 7 columns.
- Target: Price
- Features: average area income, house age, number of rooms, number of bedrooms, and population.
- No missing values were found in the dataset.

- The Address column was excluded because it contains textual information that is not useful for regression modeling.

After correlation analysis, correlations with Price are:

- Avg. Area Income (0.639734) – it is the strongest relationship.
- Avg. Area House Age (0.452543) – moderate positive correlation.
- Area Population (0.408556) – also a moderate positive correlation.
- Avg. Area Number of Rooms (0.335664) – weak to moderate positive relationship. The more rooms, the more expensive the house, but the impact is not very strong.
- Avg. Area Number of Bedrooms (0.171071) – very weak relationship. The number of bedrooms alone does little to explain the price of a home.

Model:

- **Linear Regression**
  - predicts  $y$  as a linear combination of features ( $y = w_1x_1 + \dots + b$ ), minimizing the sum of squared errors (MSE)
- Decision Tree Regressor
  - splits data based on feature conditions (e.g., if  $x > 5$ , then...), prediction = average value in the leaf.
- Random Forest Regressor
  - an ensemble of many Decision Trees trained on random subsets of data and features; final prediction = average of all trees' outputs.

Three regression models were tested: Linear Regression, Decision Tree Regressor, and Random Forest Regressor. After comparing their performance using MAE, RMSE, and  $R^2$  metrics, Linear Regression achieved the best results and was selected as the final model for the project.

Prediction Task:

Predict the sale price of a house based on area characteristics such as average income, house age, number of rooms, number of bedrooms, and population.

## Implementation Explanation

### 1. Data Loading

Read USA\_Housing.csv using pandas. No cleaning required since the dataset has no missing values and incorrect data.

```
def load_data(path: str) -> pd.DataFrame:
    return pd.read_csv(path)
```

### 2. Preprocessing

Dropped the 'Address' column because we calculate only numeric data

Split features (X) and target (y=Price)

Train/test split: 80% train, 20-%test, random state = 42

```
def prepare_data(df):
    df = df.drop("Address", axis=1)
    X = df.drop("Price", axis=1)
    y = df["Price"]
    return X, y

def main():
    df = load_data("data/USA_Housing.csv")
    perform_eda(df)
    X, y = prepare_data(df)
    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42 )
```

### 3. Training

Three models were trained and compared:

Linear Regression - finds the best linear relationship between features and price .

Decision Tree Regressor - splits data into regions based on feature thresholds.

Random Forest Regressor - ensemble of 100 decision trees , averages their predictions to reduce variance and improve generalization.

```
def train_linear_regression(X_train, X_test, y_train, y_test):
    lr = LinearRegression()
    lr.fit(X_train, y_train)
    y_pred = lr.predict(X_test)
    return lr
```

### 4. Evaluation

All three models were evaluated on the same held-out test set (1000 samples) using three metrics:

-MAE- average prediction error in dollars

-RMSE- penalizes large errors more than MAE

-R^2- proportion of variance explained by the model

```
def print_metrics(y_true, y_pred):
    print(f"MAE: {mean_absolute_error(y_true, y_pred):,.0f}")
    print(f"RMSE: {np.sqrt(mean_squared_error(y_true,
```

```
y_pred) : ,.0f} ")
    print(f"R²: {r2_score(y_true, y_pred) :.4f} ")
```

## Screenshots

The screenshot shows a Jupyter Notebook named `diagram.ipynb` open in VS Code. The notebook contains the following code:

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
```

The output of the first cell shows the execution time: `[1] 1s 328ms`.

The second cell contains the following code:

```
1 df_train = pd.read_csv("E:\aitu\edu pr\educational-practice-project-defense-main\educational-practice-project-defense-main\data\USA_Housing.csv")
2 print(df_train)
```

The output of the second cell shows the execution time: `[2] 96ms`.

A tooltip is displayed over the file path, showing the file's metadata:

- Size: 726,21 kB
- Type: CSV/TSV Data (Plain text)
- Modified: 09.06.2026 16:05
- Created: 10.06.2026 01:26

The preview of the CSV data is as follows:

|   | Avg. Area | Number of Bedrooms | Area Population | Price \ |
|---|-----------|--------------------|-----------------|---------|
| 0 | 4.09      | 23086.800503       | 1.059034e+06    |         |
| 1 | 3.09      | 40173.072174       | 1.505891e+06    |         |
| 2 | 5.13      | 36882.159400       | 1.058988e+06    |         |
| 3 | 3.26      | 34310.242831       | 1.260617e+06    |         |
| 4 | 4.23      | 26354.109472       | 6.309435e+05    |         |

The status bar at the bottom indicates the file is `diagram.ipynb` in the `src` directory of the `educational-practice-project-defense-main` project, using Python 3.13.5.

educational-practice-project-defense-main

Project

- educational-practice-project-defense-main
  - .venv library root
  - educational-practice-project-defense-main
    - github
    - idea
    - data
      - USA\_Housing.csv
    - report
      - image.png
      - REPORT\_TEMPLATE.md
    - src
      - diagram.ipynb
      - main.py
    - tests
      - README.md
      - requirements.txt
      - pyproject.toml
      - uv.lock
    - External Libraries
    - Scratches and Consoles

main.py

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
```

✓ [1] 1s 328ms

```
1 df_train = pd.read_csv("E:/aitu/edu pr/educational-practice-project-defense-main/educational-practice-project-defense-main/data/USA_Housing.csv")
2 print(df_train.head())
```

✓ [2] 96ms

|   | Avg. Area Income | Avg. Area House Age | Avg. Area Number of Rooms |  |
|---|------------------|---------------------|---------------------------|--|
| 0 | 79545.458574     | 5.682861            | 7.009188                  |  |
| 1 | 79248.642455     | 6.002900            | 6.730821                  |  |
| 2 | 61287.067179     | 5.865890            | 8.512727                  |  |
| 3 | 63345.240046     | 7.188236            | 5.586729                  |  |
| 4 | 59982.197226     | 5.040555            | 7.839388                  |  |

|   | Avg. Area Number of Bedrooms | Area Population | Price        |  |
|---|------------------------------|-----------------|--------------|--|
| 0 | 4.09                         | 23086.800503    | 1.059034e+06 |  |
| 1 | 3.09                         | 40173.072174    | 1.505891e+06 |  |
| 2 | 5.13                         | 36882.159400    | 1.058988e+06 |  |
| 3 | 3.26                         | 34310.242831    | 1.260617e+06 |  |
| 4 | 4.23                         | 26354.109472    | 6.309435e+05 |  |

educational-practice-project-defense-main > educational-practice-project-defense-main > src > diagram.ipynb 2:1 LF UTF-8 8 spaces uv (educational-practice...nse-main) [Python 3.13.5]

educational-practice-project-defense-main

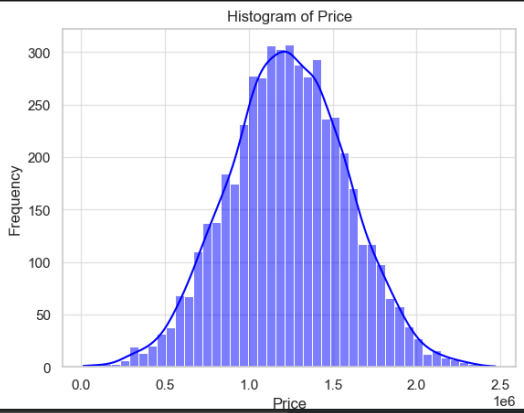
Project

- educational-practice-project-defense-main
  - .venv library root
  - educational-practice-project-defense-main
    - github
    - idea
    - data
      - USA\_Housing.csv
    - report
      - image.png
      - REPORT\_TEMPLATE.md
    - src
      - diagram.ipynb
      - main.py
    - tests
      - README.md
      - requirements.txt
      - pyproject.toml
      - uv.lock
    - External Libraries
    - Scratches and Consoles

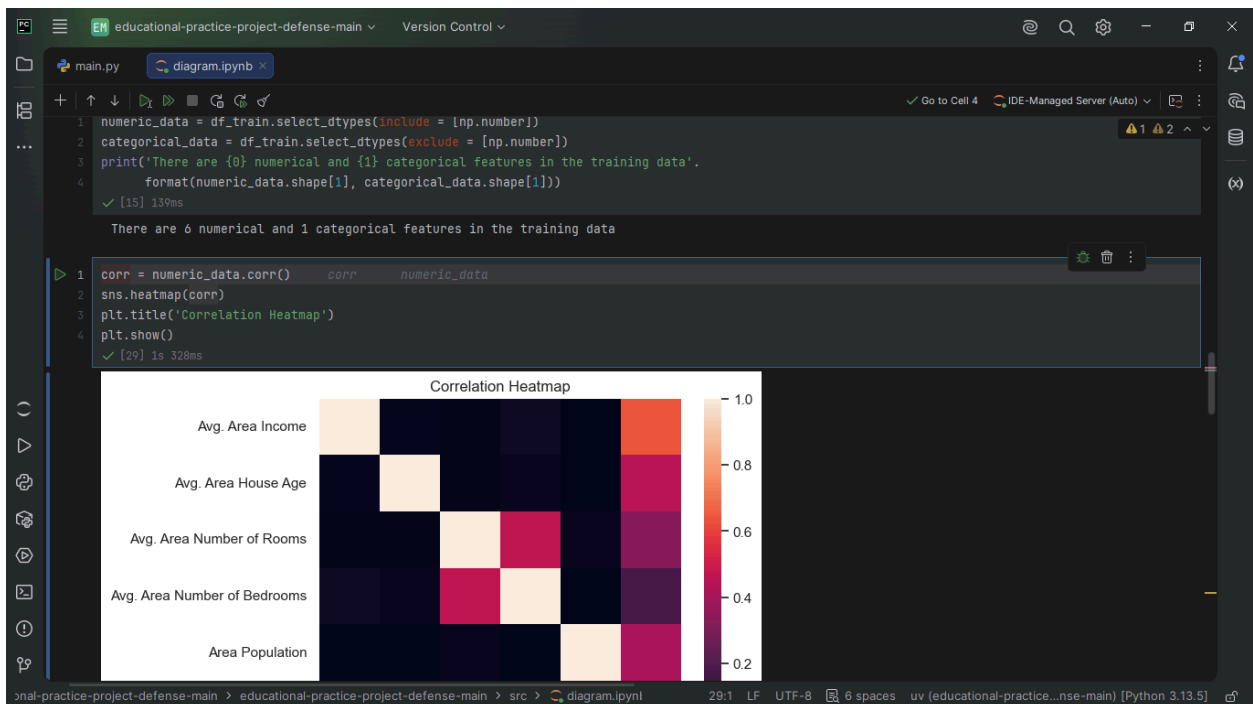
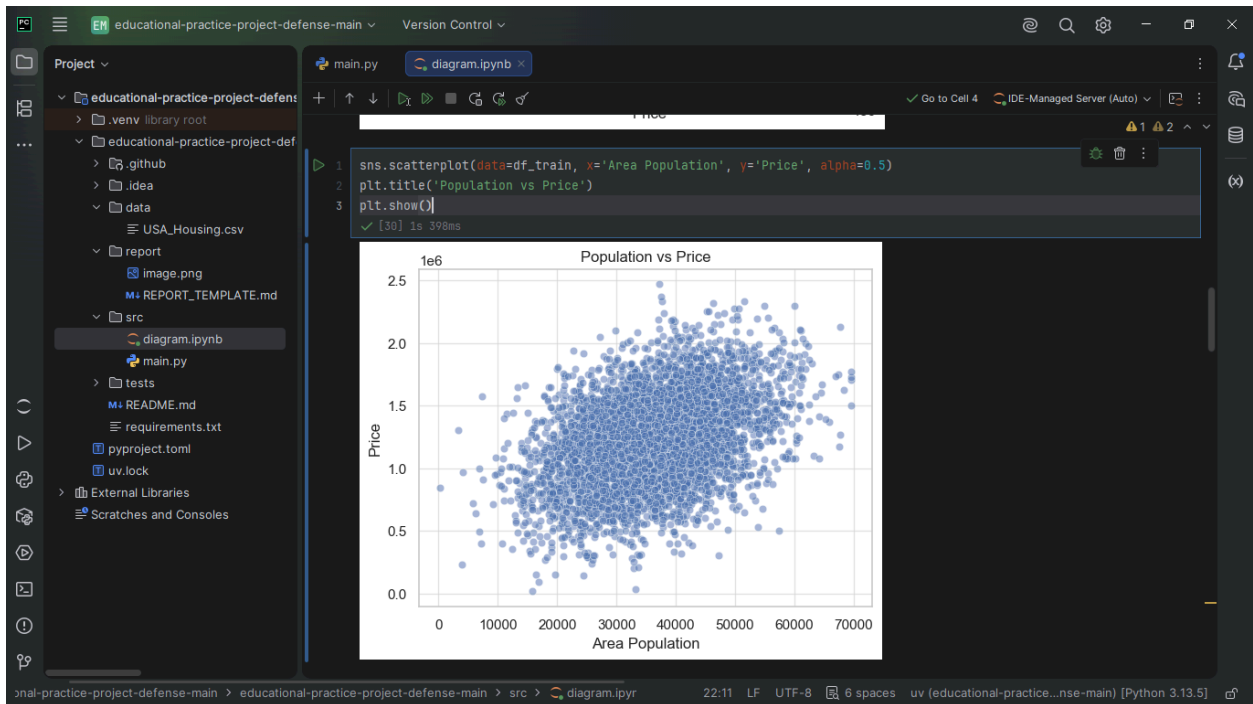
main.py

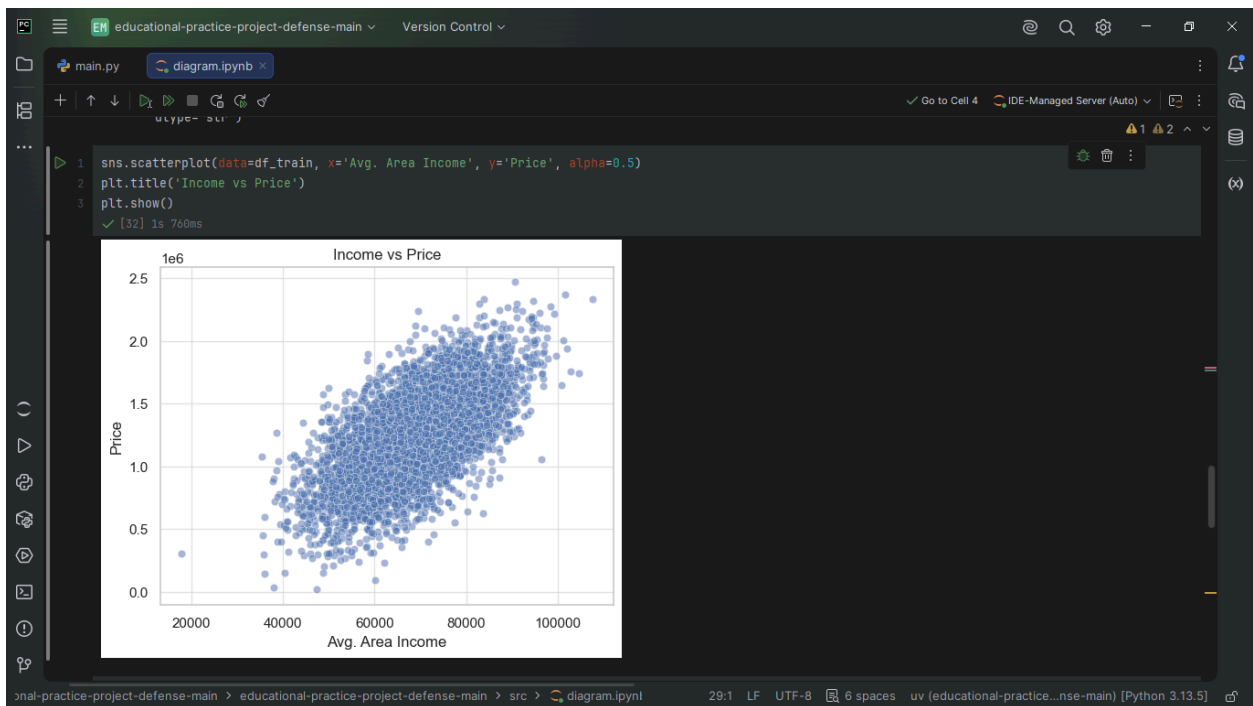
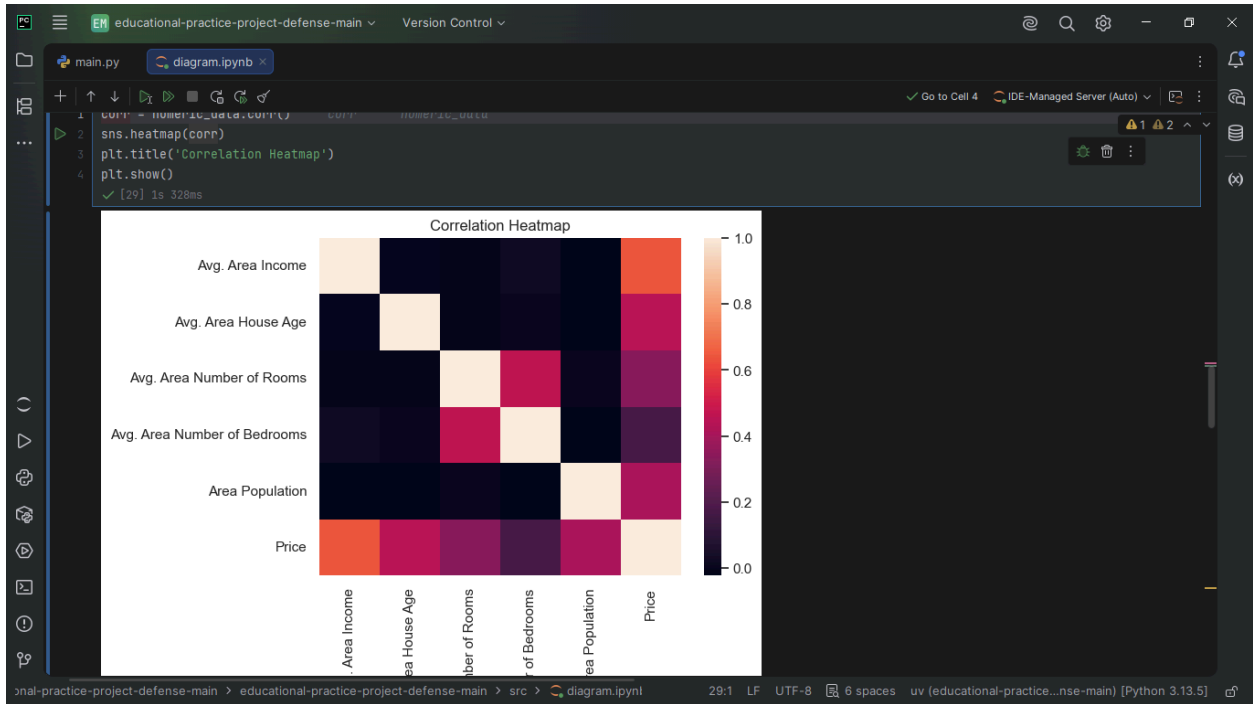
```
1 sns.histplot(df_train['Price'], kde=True, color='blue')
2 plt.title('Histogram of Price')
3 plt.xlabel('Price')
4 plt.ylabel('Frequency')
5 plt.show()
```

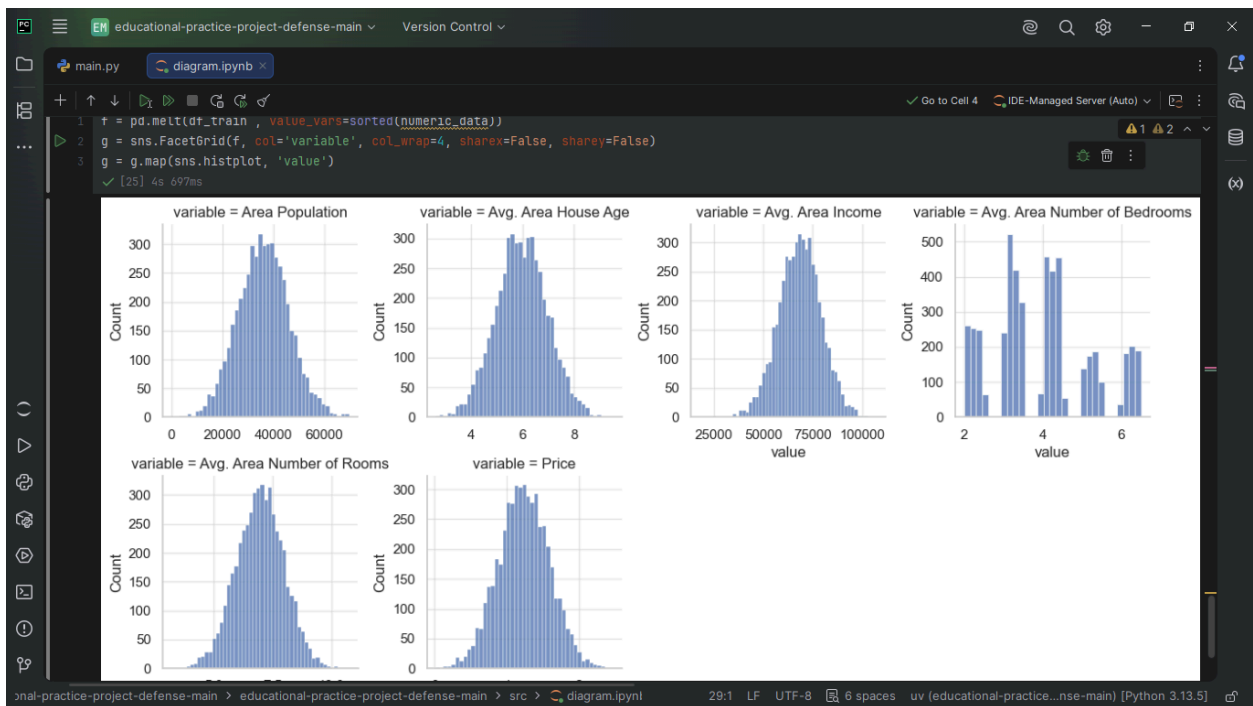
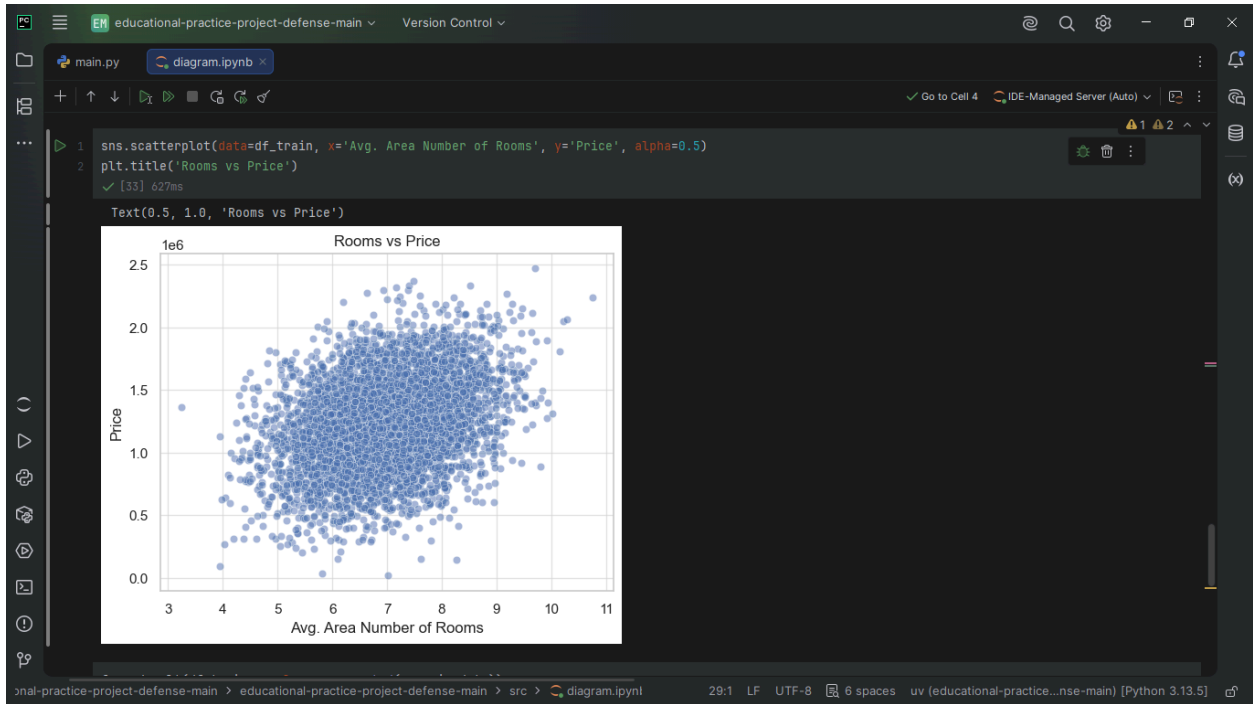
✓ [34] 1s 369ms



educational-practice-project-defense-main > educational-practice-project-defense-main > src > diagram.ipynb 2:1 LF UTF-8 8 spaces uv (educational-practice...nse-main) [Python 3.13.5]







```
df = df_train.drop(columns=['Address'])
X = df.drop(columns=['Price'])
Y = df['Price']
print(X.columns.tolist())
print(X.shape)
✓ [45] 130ms
```

['Avg. Area Income', 'Avg. Area House Age', 'Avg. Area Number of Rooms', 'Avg. Area Number of Bedrooms', 'Area Population']  
(5000, 5)

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    Y,
    test_size=0.2,
    random_state=42
)
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
✓ [47] 298ms
```

(4000, 5)  
(1000, 5)  
(4000,)  
(1000,)

```
1 from sklearn.linear_model import LinearRegression
2 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
3 lr = LinearRegression()
4 lr.fit(X_train_scaled, y_train)
5 y_pred_lr = lr.predict(X_test_scaled)
6 print("Linear Regression")
7 print(f"MAE: {mean_absolute_error(y_test, y_pred_lr):.0f}")
8 print(f"RMSE: {np.sqrt(mean_squared_error(y_test, y_pred_lr)):.0f}")
9 print(f"R^2: {r2_score(y_test, y_pred_lr):.4f}")
✓ [51] 177ms
```

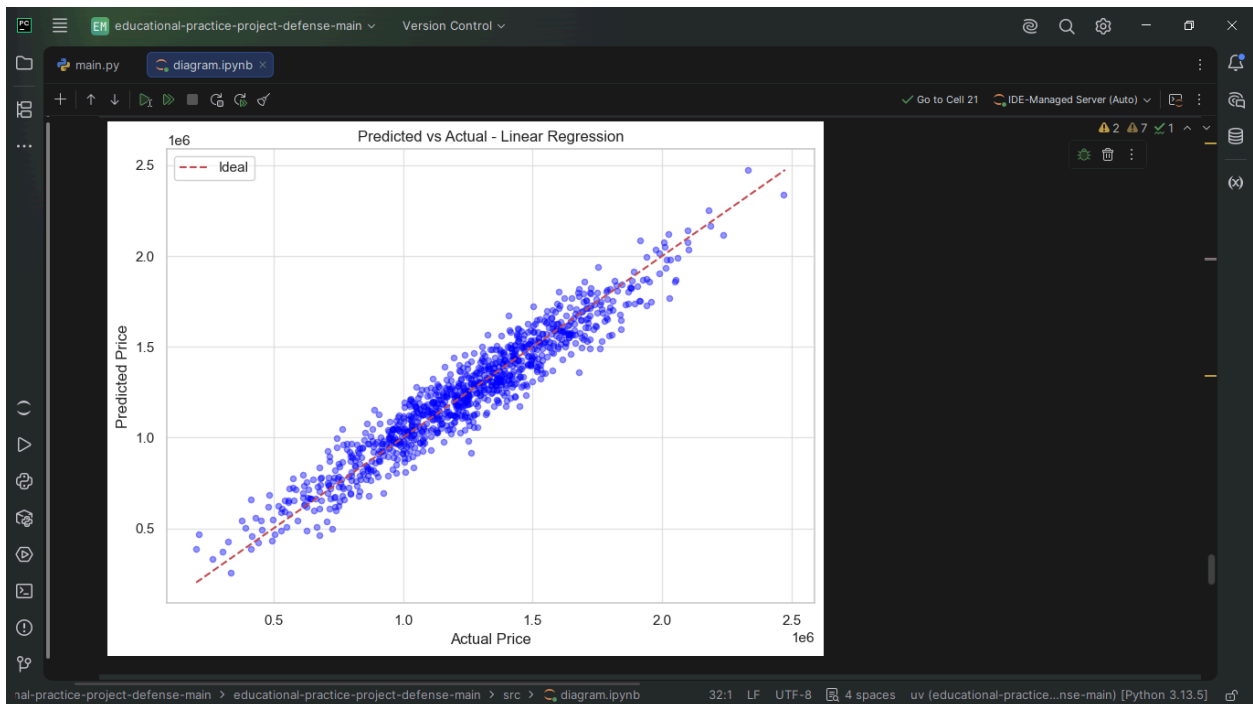
Linear Regression  
MAE: 80,879  
RMSE: 100,444  
R^2: 0.9180

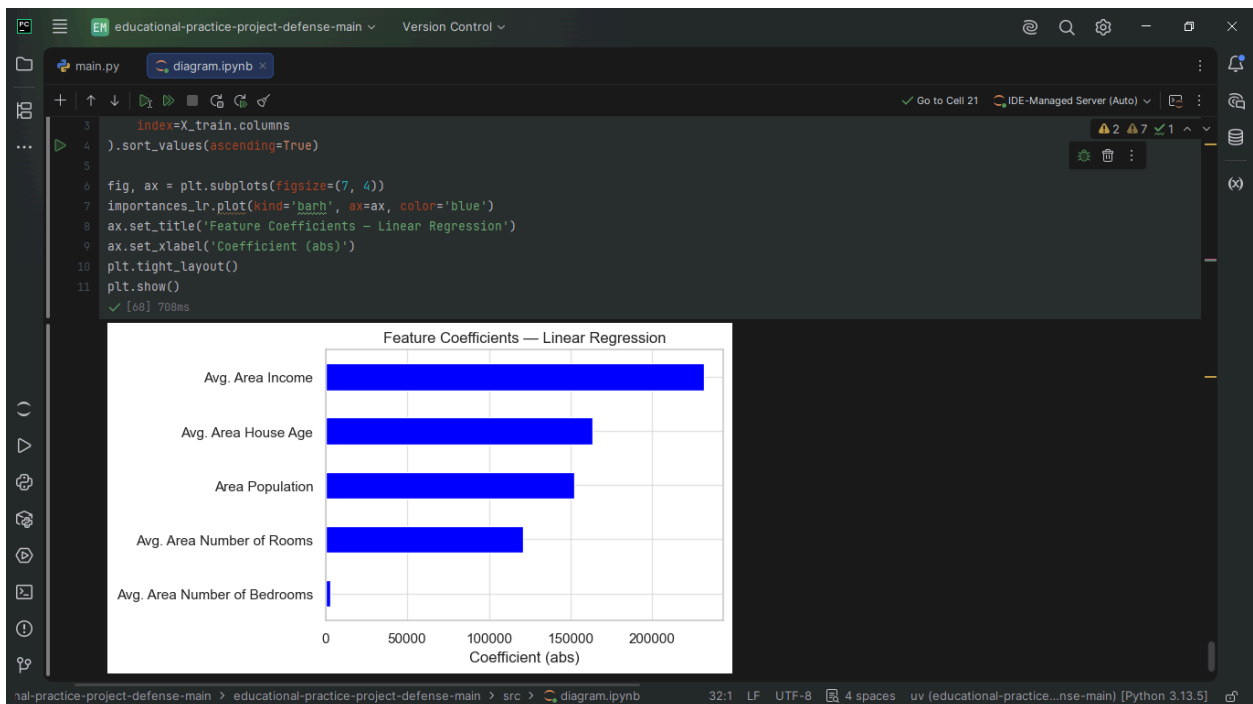
```
1 from sklearn.tree import DecisionTreeRegressor
2 dt = DecisionTreeRegressor(max_depth=10, random_state=42)
3 dt.fit(X_train_scaled, y_train)
4 y_pred_dt=dt.predict(X_test_scaled)
5 print("\nDecision Tree Regressor")
6 print(f"MAE: {mean_absolute_error(y_test, y_pred_dt):.0f}")
7 print(f"RMSE: {np.sqrt(mean_squared_error(y_test, y_pred_dt)):.0f}")
8 print(f"R^2: {r2_score(y_test, y_pred_dt):.4f}")
✓ [52] 12s 440ms
```

Decision Tree Regressor  
MAE: 133,997  
RMSE: 170,977  
R^2: 0.7624

```
1 from sklearn.ensemble import RandomForestRegressor
2 rf = RandomForestRegressor(n_estimators=100, random_state=42)
3 rf.fit(X_train_scaled, y_train)
4 y_pred_rf = rf.predict(X_test_scaled)
5 print("\n Random Forest Regressor")
6 print(f"MAE: {mean_absolute_error(y_test, y_pred_rf):.0f}")
7 print(f"RMSE: {np.sqrt(mean_squared_error(y_test, y_pred_rf)):.0f}")
8 print(f"R^2: {r2_score(y_test, y_pred_rf):.4f}")
✓ [53] 11s 881ms
```

Random Forest Regressor  
MAE: 94,645  
RMSE: 120,252  
R^2: 0.8825





```
=== House Price Prediction ===
1. Show Dataset Information
2. Train Model
3. Predict House Price
4. Exit

Select option: █
```

CLI interface

# Testing or Evaluation Results

Comparison table

| Model                   | MAE     | RMSE    | R <sup>2</sup> Score |
|-------------------------|---------|---------|----------------------|
| Linear Regression       | 80,879  | 100,444 | 0.9180               |
| Random Forest Regressor | 94,645  | 120,252 | 0.8825               |
| Decision Tree Regressor | 133,997 | 170,977 | 0.7624               |

Analysis:

- **Linear Regression** showed the best performance with the lowest MAE and RMSE values and the highest R<sup>2</sup> score
- **Random Forest Regressor** showed good results but was less accurate than Linear Regression
- **Decision Tree Regressor** showed the weakest performance among the tested models

Conclusion:

**Linear Regression** achieved the best results among all tested models. It had the lowest prediction error and the highest R<sup>2</sup> score (0.9180). Therefore, it was chosen as the final model for the project.

# Difficulties and Solutions

| Problem                                                                                                              | Solutions                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Learn the basics of machine learning, including regression models and evaluation metrics                             | We find some free playlist on youtube and also read some chapters from Aurélien Géron - Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow 3rd Edition |
| Issues with Python libraries                                                                                         | Creating virtual environment (.venv)                                                                                                                                   |
| Managing dataset file paths and project structure. In linux operation system it locate differently that on windows . | We don't use full path data/USA_Housing.csv                                                                                                                            |

## Conclusion

In this project we built a machine learning program to predict house prices using the USA Housing dataset.

- Completed full EDA with 5 visualizations revealing that Avg. Area Income is the strongest predictor
- Implemented preprocessing program with proper train/test split
- Trained and compared 3 regression models : Linear Regression , Decision Tree, Random Forest
- Best model (Linear Regression) achieved  $R^2 = 0.9180$  , MAE = 80,879 \$
- All code is structured into reusable functions in main.py and documented in diagram.ipynb

Possible future improvements : feature engineering from Address and testing Gradient Boosting or XGBoost

## Links

GitHub repository link:

<https://github.com/aitu-educational-practice-2026/educational-practice-project-defense>

Demo video link:

<https://www.youtube.com/watch?v=JYSsPkXzIVM>